

# CNT 4419 Secure Coding

## Project 1: TLS Client Implementation

Name: Junweifeng

Date: March 3, 2025

### Implementation of the TLS Client

The TLS client is implemented in Python using the socket and SSL libraries to establish a secure connection with the server. The client first creates a TCP connection using the `socket.create_connection()` function. It then initializes an SSL context with `SSL.create_default_context()`, specifying the root CA certificate file (`root_certificate.crt.pem`) for authentication. Once the TLS handshake is performed, the client verifies that the server meets the required security standards. Specifically, it checks whether the server certificate is signed by the trusted root CA, whether the certificate is still valid and has not expired, and whether the server enforces forward secrecy by supporting secure key exchange algorithms such as ECDHE or DHE. If the server fails to meet these security requirements, the client prints an appropriate error message and terminates the connection. However, if all checks pass, the client sends a test message to the server and prints the response. To facilitate execution, a shell script (`tls_client.sh`) was created, allowing the user to run the client by simply providing the server's IP address and port. Additionally, a Makefile is included to handle compilation if needed, and it provides a cleanup command to remove temporary files. During the development process, I consulted various online resources to understand and troubleshoot socket programming, SSL/TLS security, and OpenSSL functions. Beej's Guide to Network Programming helped me gain a solid understanding of TCP socket programming, particularly in establishing and managing a connection between a client and a server in Python. Additionally, I referred to the OpenSSL 3.0 API Documentation to learn how SSL/TLS functions work in Python, including setting up an SSL context, performing a TLS handshake, and verifying certificates. The Python SSL Library Documentation was also instrumental in helping me correctly implement certificate validation and secure TLS configurations. Furthermore, I found Zakird's Guide to Certificate Parsing useful for understanding how to read and validate X.509 certificates, which was crucial for ensuring the correctness of my implementation. In addition to these online resources, I used ChatGPT to assist with debugging errors, refining implementation details, and clarifying OpenSSL functions. When setting up the TLS client, I asked how to create a TLS client in Python using OpenSSL, which provided a structured approach to using the socket and SSL libraries. I also encountered an issue with certificate expiration validation and sought guidance on how to check if an SSL certificate had expired in Python. ChatGPT suggested using `DateTime.strptime()` to parse the certificate's `notAfter` field and compare it with the current time. While this advice was helpful, I later found that `datetime.utcnow()` was deprecated and adjusted my approach accordingly. Another challenge I faced was enforcing forward secrecy in TLS 1.2, and ChatGPT provided guidance on identifying the necessary key exchange algorithms (ECDHE/DHE) to ensure forward secrecy.

Additionally, when troubleshooting a `CERTIFICATE_VERIFY_FAILED` error, ChatGPT suggested checking the root CA certificate path and ensuring my client used `it.create_default_context(cafile="root_certificate.crt.pem")` for proper authentication. This helped me identify and resolve the issue, allowing the client to correctly validate certificates. Although ChatGPT provided useful insights, I made sure to verify all AI-generated suggestions against official OpenSSL and Python documentation to ensure accuracy and adherence to security best practices. The combination of AI assistance, online documentation, and manual verification allowed me to successfully implement a secure and fully functional TLS client.

## Findings Regarding Server Instances

The TLS client was tested against three server instances at IP address 10.10.11.145, with ports 9000, 9090, and 9099. Each server was analyzed to determine whether it met the specified security requirements: certificate validation, certificate expiration, and forward secrecy.

1. Server at Port 9000:
  - a. Output: "Requirement 3 not met."
  - b. Analysis: This indicates that the server does not enforce forward secrecy. The cipher suite used does not include ECDHE or DHE key exchange methods, which are necessary for ensuring forward secrecy in TLS 1.2. Since this requirement is not met, the client correctly terminates the connection.
2. Server at Port 9090:
  - a. Output: "Requirement 3 not met."
  - b. Analysis: Similar to the previous case, this server does not enforce forward secrecy. The TLS handshake reveals that the server is using a cipher suite that lacks the necessary key exchange mechanisms to guarantee forward secrecy.
3. Server at Port 9099:
  - a. Output: " Requirement 2 not met ."
  - b. Analysis: This error indicates that the server's certificate has expired. The client correctly identifies this issue and terminates the connection as per the requirement that the certificate must be valid and not expired. This confirms that requirement 2 was not met for this server instance.

```

try:
    # Create a TCP connection to the server
    sock = socket.create_connection((server_ip, server_port))

    # Set up TLS context with root CA
    context = ssl.create_default_context(cafile=ca_cert_path)
    context.minimum_version = ssl.TLSVersion.TLSv1_2 # Enforce TLS 1.2 or higher
    context.check_hostname = False # Disable hostname verification

    # Wrap socket in SSL/TLS
    tls_conn = context.wrap_socket(sock, server_hostname=server_ip)

    # Get server certificate
    cert = tls_conn.getpeercert()

    # Check if the certificate is available
    if not cert:
        print("Requirement 1 not met.", file=sys.stderr)
        sys.exit(1)

    # Check if the certificate is expired
    if "notAfter" in cert:
        expiry_date = datetime.datetime.strptime(cert['notAfter'], "%b %d %H:%M:%S %Y %Z")
        expiry_date = expiry_date.replace(tzinfo=pytz.UTC) # Convert to timezone-aware datetime
        # Compare expiration date with current UTC time
        if expiry_date < datetime.datetime.now(pytz.UTC):
            print("Requirement 2 not met.", file=sys.stderr)
            sys.exit(1)
    else:
        print("Error: Certificate does not contain 'notAfter' field.", file=sys.stderr)
        sys.exit(1)

    # Check forward secrecy (TLS 1.2 with ECDHE/DHE key exchange)
    cipher = tls_conn.cipher()[0] # Get the cipher suite used in the connection
    if "ECDHE" not in cipher and "DHE" not in cipher:
        print("Requirement 3 not met.", file=sys.stderr)
        sys.exit(1)

    # Send a message to the server
    tls_conn.sendall(b"Hello, Server!")

    # Receive response
    response = tls_conn.recv(1024)

```

```

(junweifeng@kali)-[~/tls_project]
└─$ ./tls_client.sh 10.10.11.145 9000
./tls_client.sh 10.10.11.145 9090
./tls_client.sh 10.10.11.145 9099

Requirement 3 not met.
Requirement 3 not met.
Requirement 2 not met.

```

I had a few problems when running my TLS client script.

First, I got an error about the datetime comparison because the certificate's expiration date was not in the same format as the current time. To fix this, I changed the code to make the expiration date timezone-aware by adding `expiry_date = expiry_date.replace(tzinfo=pytz.UTC)`. I also replaced `datetime.datetime.utcnow()` with `datetime.datetime.now(pytz.UTC)`, so both times had

the same format. Second, I had an IndentationError because the spacing in my script was not consistent. Some lines used tabs, and others used spaces, which caused Python to throw an error. To fix this, I made sure all indentation was the same by using spaces everywhere. Lastly, I got a connection refused error, which happened because my client could not reach the server. This wasn't a coding issue but a setup issue. To fix it, I checked my VPN connection, made sure I was using the correct IP and port, and verified that the server was running before trying again. After making these changes, my TLS client worked correctly, and I was able to check the server's security requirements

```
./tls_client.sh 10.10.11.145 9090
./tls_client.sh 10.10.11.145 9099

Error: [Errno 111] Connection refused
^CTraceback (most recent call last):
  File "/home/junweifeng/tls_client.py", line 14, in <module>
    sock = socket.create_connection((server_ip, server_port))
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/socket.py", line 850, in create_connection
    sock.connect(sa)
KeyboardInterrupt

(junweifeng@kali)-[~]
└─$ ./tls_client.sh 10.10.11.145 9000
./tls_client.sh 10.10.11.145 9090
./tls_client.sh 10.10.11.145 9099

Error: can't compare offset-naive and offset-aware datetimes
Error: can't compare offset-naive and offset-aware datetimes
TLS Error: [SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: certificate has expired (_ssl.c:1000)

(junweifeng@kali)-[~]
└─$ nano tls_client.py

(junweifeng@kali)-[~]
└─$ ./tls_client.sh 10.10.11.145 9000
./tls_client.sh 10.10.11.145 9090
./tls_client.sh 10.10.11.145 9099

File "/home/junweifeng/tls_client.py", line 27
    expiry_date = expiry_date.replace(tzinfo=pytz.UTC) # Convert expiry_date to timezone-aware
    ^
IndentationError: unindent does not match any outer indentation level
File "/home/junweifeng/tls_client.py", line 27
    expiry_date = expiry_date.replace(tzinfo=pytz.UTC) # Convert expiry_date to timezone-aware
    ^
IndentationError: unindent does not match any outer indentation level
File "/home/junweifeng/tls_client.py", line 27
    expiry_date = expiry_date.replace(tzinfo=pytz.UTC) # Convert expiry_date to timezone-aware
    ^
IndentationError: unindent does not match any outer indentation level

(junweifeng@kali)-[~]
└─$ ls -l root_certificate.crt.pem

-rw-rw-r-- 1 junweifeng junweifeng 908 Mar  2 17:23 root_certificate.crt.pem
```

```

GNU nano 8.2                                     tls_client.py
import socket # Import socket module for network communication
import ssl    # Import SSL module for secure connection
import sys    # Import sys module for command-line argument handling
import datetime # Import datetime for handling expiration dates
import pytz   # Import pytz for timezone-aware datetime handling

# Ensure correct number of arguments
if len(sys.argv) != 3: # Checks if the number of arguments is not exactly 3
    print("Usage: python3 tls_client.py <server_IP> <server_PORT>")
    sys.exit(1) # Exit the program with an error code

# Retrieve command-line arguments
server_ip = sys.argv[1] # The first argument is the server IP
server_port = int(sys.argv[2]) # The second argument is the server port (convert to int)

# Define the path to the provided root CA certificate
ca_cert_path = "root_certificate.crt.pem" # Use your actual root CA file

try:
    # Create a TCP connection to the server
    sock = socket.create_connection((server_ip, server_port))

    # Set up TLS context with root CA
    context = ssl.create_default_context(cafile=ca_cert_path)
    context.minimum_version = ssl.TLSVersion.TLSv1_2 # Enforce TLS 1.2 or higher
    context.check_hostname = False # Disable hostname verification

    # Wrap socket in SSL/TLS
    tls_conn = context.wrap_socket(sock, server_hostname=server_ip)

    # Get server certificate
    cert = tls_conn.getpeercert()

    # Check if the certificate is available
    if not cert:
        print("Requirement 1 not met.", file=sys.stderr)
        sys.exit(1)

    # Check if the certificate is expired
    if "notAfter" in cert:
        expiry_date = datetime.datetime.strptime(cert['notAfter'], "%b %d %H:%M:%S %Y %Z")
        expiry_date = expiry_date.replace(tzinfo=pytz.UTC) # Convert to timezone-aware datetime
        # Compare expiration date with current UTC time
        if expiry_date < datetime.datetime.now(pytz.UTC):
            print("Requirement 2 not met.", file=sys.stderr)

```